ZRust User Manual

Version 0.1.0 · Source use on Apple Silicon macOS

This manual explains how to build and run `zrust`. No compiled executable is supplied. The examples use small synthetic counts. They do not represent an experiment or establish a biological result.

Keep this manual beside your source folder. Type commands in Terminal. Do not type the surrounding code-block borders. Run each numbered step in order. Stop if a command reports an error. Read section 20 before you continue.

Contents

1. What ZRust does · 2. System requirements

3. Open Terminal · 4. Install Rust

5. Obtain ZRust · 6. Build ZRust

7. Confirm the installation · 8. Count-matrix format

9. Check sample names · 10. First paired analysis

11. First unequal-arm unpaired analysis · 12. Choose samples

13. Main command options · 14. Thread settings

15. Gene-level output · 16. Guide-level output

17. Sensitization and resistance · 18. P-values and FDR

19. Safe output writing · 20. Common errors and fixes

21. Reproducibility · 22. Current limits

23. Complete command reference · 24. Worked examples

The repository provides the manuscript and this manual. Repository access is required while it is private.

1. What ZRust does

ZRust compares guide counts from control and treatment samples in pooled CRISPR screens. A guide directs a perturbation toward a gene. A count records how often that guide was observed in a sample. A count matrix is a table of these values.

ZRust is an independent Rust reimplementation of DrugZ. Rust is the programming language used to build the program. DrugZ combines guide changes into gene-level scores. The results describe sensitization and resistance to a treatment. Sections 17 and 18 explain these terms.

ZRust reads counts that you already have. It does not count sequencing reads. It does not select samples for you. It does not check whether an experiment has adequate replication or biological controls.

The project name is ZRust. The crate, executable, and command are named `zrust`. A crate is a Rust software package. An executable is a program you can run. The current version is 0.1.0.

2. System requirements

Use an Apple Silicon Mac. Open the Apple menu and select About This Mac. The chip name must identify Apple Silicon, such as an Apple M-series chip. Source use is qualified only for macOS ARM64, also called the Rust target `aarch64-apple-darwin`.

You need Git, Rust, Cargo, and Apple command-line developer tools. Git obtains the source files. Cargo builds and tests Rust software. Apple developer tools provide the system tools needed to link the executable. Internet access is needed to obtain source and uncached dependencies.

Rust/Cargo 1.98.0 was tested. A lower minimum version has not been established. Section 4 gives a setup route for that version. Python 3 is needed for the fixture-integrity test and the optional verification command. Python scientific packages are not needed. Python is not a runtime requirement for ZRust analysis.

The largest tested input had one million guides. No general minimum memory specification has been established. Allow space for source, downloaded dependencies, compiler output, input data, and result files.

3. How to open Terminal

1. Press Command and Space.

2. Type Terminal.

3. Press Return.

A window opens with a prompt. The prompt shows that the shell is ready for a command. A shell reads the commands you type. Do not copy a prompt character such as `$` into a command.

Run this check:

```
uname -m
git --version
xcode-select -p
```

Expected results: `arm64`, a Git version, and a path for Apple developer tools. If the tools are missing, open the Mac's developer-tool installation prompt when macOS offers it. Complete Apple's installation before continuing. If no prompt appears, use Apple's official command-line tools instructions.

A command can print several lines. Wait until the prompt returns before you enter the next command. A successful quiet analysis can print nothing.

4. How to install Rust

If `rustc --version` and `cargo --version` already work with version 1.98.0, keep that installation and go to section 5. Do not replace a managed workplace installation without local approval.

The following route uses the official Rust installer. The first command downloads its script. The second runs it. The `--no-modify-path` option leaves your shell startup files unchanged. The next command enables Rust in this Terminal window.

1. Download the installer.

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs -o rustup-init.sh
```

2. Run the installer.

```
sh rustup-init.sh -y --profile minimal --no-modify-path
```

3. Enable the installed commands in this window.

```
. "$HOME/.cargo/env"
```

4. Install the tested toolchain.

```
rustup toolchain install 1.98.0 --profile minimal
```

5. Select that toolchain.

```
rustup default 1.98.0
```

6. Add the formatter.

```
rustup component add rustfmt
```

This selects 1.98.0 for your Rust projects. On a shared or managed setup, ask the administrator to provide that toolchain instead. `rustfmt` is the formatter used by the optional source checks. Open a new Terminal window and repeat step 3 if the commands are not found.

5. How to obtain ZRust

Use the existing ZRust GitHub repository. It requires authorized GitHub access while it is private. Use your normal approved GitHub authentication. Do not put a token or password in a command, input file, or issue report.

1. Start in a directory where you can create a new folder named `zrust`.

2. Run the clone command.

```
git clone https://github.com/Yoonecorn/ZRust.git zrust
```

3. Enter the new folder.

```
cd zrust
```

Expected result: the source folder contains `Cargo.toml`, `Cargo.lock`, `src`, `tests`, `examples`, `docs`, and `manuscript`. Keep all supplied support directories. Tests depend on their paths.

A relative path starts from your current directory. All remaining analysis commands assume that you are in this repository root. A repository root is the folder that contains `Cargo.toml`.

6. How to build ZRust

Building converts source files into an executable. Cargo uses `Cargo.lock` to select exact dependency versions. `--locked` prevents it from changing that selection.

1. Clear a custom compiler-output location for this shell.

2. Build the executable.

```
unset CARGO_TARGET_DIR
cargo build --release --locked
```

Expected result: Cargo finishes the release build without an error. The executable is `target/release/zrust`. The first build can take longer because Cargo downloads dependencies. This download is normal. Dependencies are not supplied as vendored source.

`target` contains generated build files. Keep it out of the source you share. Do not move the executable alone and assume that other machines are supported. No binary distribution is provided.

7. How to confirm the installation

1. Check the compiler.

2. Check Cargo.

3. Check ZRust.

```
rustc --version
cargo --version
target/release/zrust --version
```

Expected ZRust result: `ZRust 0.1.0`. The compiler and Cargo should report 1.98.0 for the tested setup.

4. Open command help.

```
target/release/zrust --help
```

The help lists options beginning with a hyphen. An option changes how a program runs. An argument supplies its value. For example, `-i counts.tsv` uses `-i` as the option and `counts.tsv` as its argument.

8. Count-matrix format

Use UTF-8 tab-separated text, often named with `.tsv`. UTF-8 is the text encoding. Each tab separates two columns. Each new line starts a row. Unix-style (LF) and Windows-style (CRLF) line endings are accepted. Do not use embedded tabs or new lines inside a label. Comma-separated values (CSV), Excel workbooks, and compressed files are not direct inputs.

Column	Content
First	A nonempty, unique guide identifier.
Second	A nonempty gene name. It can repeat for guides of the same gene.
Remaining	A unique, nonempty sample name in the header, then counts.

Use actual tab characters, not aligned spaces. The supplied `examples/paired/counts.tsv` is a real TSV file with 80 guides and 20 synthetic genes. Its samples are `C1`, `C2`, `T1`, and `T2`.

Selected counts must be finite and nonnegative. Finite means neither infinite nor undefined. Selected sample totals must be positive after excluded genes are removed. ZRust accepts finite fractional counts, but use the count data appropriate for your study. Do not invent values to bypass an input error.

Every row must have the same number of columns as the header. A blank line is not a comment. ZRust does not interpret quoted spreadsheet fields or comment lines.

9. How to check sample names

1. Read the header of the teaching input.

```
head -n 1 examples/paired/counts.tsv
```

Expected header: `sgRNA`, `Gene`, `C1`, `C2`, `T1`, `T2`, separated by tabs. Copy sample names exactly, including capital letters. Sample names are case-sensitive.

A sample list uses literal commas. Use `C1,C2`, not `C1, C2`. Spaces after a comma become part of a sample name. A sample must appear in only one selected group. Do not repeat a sample within a group.

10. First paired analysis

A paired analysis matches controls and treatments by list position. In this teaching example, `C1` pairs with `T1`, and `C2` pairs with `T2`. Both groups must have the same nonzero number of samples. Confirm that this pairing represents your experiment before using real data.

1. Create a new output folder.

2. Run the analysis.

3. View the gene header and first two result rows.

```
mkdir results-first-paired
target/release/zrust -i examples/paired/counts.tsv \
  -c C1,C2 -x T1,T2 --half_window_size 5 -q \
  -o results-first-paired/genes.tsv \
  -f results-first-paired/guides.tsv
head -n 3 results-first-paired/genes.tsv
```

Expected result: two complete files and a returned prompt. The gene output starts with the header described in section 15. Its first gene is `G18`, with `normZ` of `-2.18` in this teaching example.

The backslash continues a command on the next line. Keep it as the last character on that line. Do not add a space after it. Copy the entire command block.

The example needs `--half_window_size 5` because it is small. The default is 500. A half-window controls the local group of guides used to estimate variation. A smaller value is not always valid under the reference method. Do not choose a value only because it makes an error disappear.

11. First unequal-arm unpaired analysis

An unpaired analysis compares mean normalized counts within each group. It does not match samples by their positions. The groups can have different sizes. This teaching example has two controls and one treatment.

1. Create a new output folder.

2. Run the unpaired analysis.

3. View the first gene rows.

```
mkdir results-first-unpaired
target/release/zrust -i examples/unpaired/counts.tsv \
  -c C1,C2 -x T1 -unpaired --half_window_size 5 --minobs 1 -q \
  -o results-first-unpaired/genes.tsv \
  -f results-first-unpaired/guides.tsv
head -n 3 results-first-unpaired/genes.tsv
```

Expected result: gene and guide files. The first gene is `G18`, with `normZ` of `-3.10`. This differs from the paired result because the sample selection and analysis are different.

`--minobs 1` keeps genes with at least one nonzero guide observation. One unpaired group comparison supplies one observation per guide with a nonzero score. This is not a count of biological replicates.

12. How to choose control and treatment samples

Start with the experimental plan. A control is the reference condition. A treatment is the condition compared with it. Do not choose samples based on which result looks strongest.

For paired analysis, put matched samples in the same list position. For unpaired analysis, select the samples whose group means should be compared. Both groups must be nonempty. Unequal sizes alone do not establish that unpaired analysis is appropriate for your study.

Unused columns can remain in the input. Selected columns must satisfy the numerical checks. Keep sample names unique across the whole header. Gene exclusions use `-r` with exact gene names. Exclusions occur before sample normalization. Normalization adjusts counts to a common selected-sample total. A pseudocount is a small added value used in the fold-change calculation; `-p` sets it after count normalization.

13. Main command options

Always supply `-c` and `-x` for analysis. Use `-i` to choose the count file. Use `-o` for gene results and `-f` for optional guide results. Use a distinct path for each file.

Purpose	Option and default
Select input	`-i FILE`; otherwise standard input.
Select groups	`-c LIST` and `-x LIST`; both required.
Write genes	`-o FILE`; otherwise standard output.
Write guides	`-f FILE`; omitted unless requested.
Change mode	`-unpaired`; otherwise paired.
Add a pseudocount	`-p INTEGER`; default 5, minimum 0.
Set local half-window	`--half_window_size INTEGER`; default 500.
Require observations	`--minobs INTEGER`; default 1.
Exclude genes	`-r LIST`; no exclusions by default.
Hide progress	`-q`; errors remain visible.

Standard input receives text from another command or a file redirect. Standard output prints text to Terminal unless redirected. These are optional advanced routes. Beginners should use explicit input and output paths as in the worked examples.

14. Thread settings

A CPU (central processing unit) worker is a thread that can prepare part of a result file. ZRust uses eight formatting workers by default and at most eight. Scientific calculations remain single-threaded. Workers only prepare completed rows and join them in the original order.

`--threads 1` requests serial output preparation. `--threads 2`, `--threads 4`, and `--threads 8` set worker limits. Positive values above eight are capped at eight. Zero is an error. Small outputs remain serial below 25,000 gene rows or 50,000 guide rows, even if more workers are requested.

The following run requests one worker. Its small input will give the same result bytes as the default run.

```
mkdir results-one-worker
target/release/zrust -i examples/paired/counts.tsv -c C1,C2 -x T1,T2 \
  --half_window_size 5 --threads 1 -q \
  -o results-one-worker/genes.tsv -f results-one-worker/guides.tsv
cmp results-first-paired/genes.tsv results-one-worker/genes.tsv
cmp results-first-paired/guides.tsv results-one-worker/guides.tsv
```

Expected result: `cmp` prints nothing and exits successfully. `cmp` compares file bytes. More workers do not guarantee a shorter run. Do not interpret this small teaching run as a performance test.

15. Gene-level output

The gene file always has these ten columns. A row describes one retained gene. The rows are sorted by `normZ` from low to high. Exact score ties use a fixed order based on the encoded gene names.

Column	Meaning
`GENE`	Gene label from the input.
`sumZ`	Sum of guide Z-scores across selected comparisons.
`numObs`	Number of nonzero guide observations used for the gene.
`normZ`	Gene score standardized across retained genes.
`pval_synth`	Lower-tail p-value for sensitization.
`rank_synth`	Sensitization rank, with the most negative gene first.
`fdr_synth`	Sensitization false discovery rate adjustment.
`pval_supp`	Upper-tail p-value for resistance or suppression.
`rank_supp`	Resistance rank, with the most positive gene first.
`fdr_supp`	Resistance false discovery rate adjustment.

A Z-score expresses a change relative to estimated variation. ZRust combines guide Z-scores, divides by the square root of the observation count, and standardizes across retained genes. Read the method paper and manuscript for the calculation details.

`numObs` is not the number of biological replicates. A guide with a score of zero does not add to this count. `--minobs` filters this count before final gene standardization. Fewer than two retained genes cause an error because cross-gene standardization is not defined.

The written score and probability values are rounded. Use full input provenance and the program version to reproduce a result. Do not expect a displayed rounded score to reproduce every internal calculation.

The teaching example's first paired row has gene `G18`, `sumZ` of `-11.03`, `numObs` of `8`, and `normZ` of `-2.18`. These are teaching outputs, not evidence about a biological gene.

16. Guide-level output

Guide output is optional. Add `-f FILE` when you need it. Paired and unpaired guide files have different columns. Do not combine them as if they had the same schema. A schema is a defined set of columns.

Paired guide output

The first column contains guide identifiers. Its header cell is intentionally empty. The second column is `GENE`. For each paired comparison, four columns follow:

Column	Meaning
Control sample name, such as `C1`	Raw count in that control sample.
`eb_std_0`	Estimated local variation for comparison 0.
`zscore_fc_0`	Guide Z-score for comparison 0.
`fc_0`	Base-two log fold change for comparison 0.

The next pair uses suffix `_1`, then `_2`, and so on. Comparison numbering starts at zero. The output rows follow the first comparison's control-abundance order, not input order. The example has ten columns in total.

Unpaired guide output

The unpaired file has exactly nine named columns:

Column	Meaning
`guide_id`	Guide identifier.
`GENE`	Gene label.
`raw_control_mean`	Mean raw count among selected controls.
`raw_treatment_mean`	Mean raw count among selected treatments.
`normalized_control_mean`	Mean normalized control count, before pseudocount addition.
`normalized_treatment_mean`	Mean normalized treatment count, before pseudocount addition.
`eb_std`	Estimated local variation.
`zscore`	Guide Z-score.
`mean_fc`	Base-two log ratio of group mean normalized counts with pseudocounts.

`mean_fc` is not the arithmetic mean of individual log fold changes. The file uses raw-control-mean order. The normal analysis does not require Python or an external sorting command.

17. Sensitization and resistance

A negative gene `normZ` means that its guides decreased more in treatment than in control. This points toward sensitization under the analysis model. A positive score points toward resistance or suppression of the treatment response. The direction depends on how you assigned controls and treatments.

Read the matching directional statistics. For a sensitization question, use `pval_synth` and `fdr_synth`. For a resistance question, use `pval_supp` and `fdr_supp`. Do not select the smaller result after looking at both directions without considering the study plan.

A statistical association is not proof of a biological mechanism. Follow-up experiments, adequate controls, and a suitable design remain necessary. ZRust does not evaluate treatment mechanisms on its own.

18. P-values and FDR

A p-value measures how extreme the observed score is under the program's statistical model. It is not the probability that a gene has no effect. A small value does not prove that a result will replicate.

FDR means false discovery rate. ZRust applies the Benjamini-Hochberg adjustment separately to sensitization and resistance p-values. The `fdr_synth` and `fdr_supp` columns hold the adjusted values. These values depend on the retained set of tested genes and the model assumptions.

Choose an interpretation threshold according to the study plan. This manual does not prescribe a universal threshold. Do not treat a displayed zero or a rounded value as an exact unrounded probability. Do not infer equivalence from a high correlation alone.

Published software comparisons used the SUM149PT olaparib screen, HeLa olaparib screen, and hTERT-RPE1 olaparib screen. The first and third matched across tested result fields. In the HeLa olaparib screen, OR4D10 and RPL7 had equal scores but opposite adjacent rank order. Their statistical results and significant-gene membership were unchanged. These are software comparisons, not new biological findings.

19. Safe output writing

ZRust checks the analysis before preparing output files. It first writes each result to a temporary file beside the intended destination. It moves that file into its final location only after the write is complete.

This protection applies to each file separately. ZRust can replace an existing destination. It does not ask whether an old file should be overwritten. Always use a new output folder when you want to keep old results.

If both files are requested, ZRust completes guide-file publication before gene-file publication. The files do not form a two-file transaction. A late gene-file failure can leave a complete new guide file. Check both outputs and the command exit status before using the results. This behavior does not guarantee recovery from a power failure or forced process termination.

Do not choose the same gene and guide destination. The program rejects aliases that it can resolve through their parent directories. It does not create missing parent directories. Create the output folder first.

When gene results go to standard output, a file's safe-writing protection does not apply to Terminal or an ordinary shell redirect. Use `-o FILE` for the file behavior described here.

After a failure, keep the input and error message. Do not interpret a file merely because it exists. Use a fresh output directory for the corrected run. Review whether an earlier guide file was replaced before a later failure.

20. Common errors and fixes

These messages come from the actual command interface or analysis checks. Words in quotes are examples of message fragments. The program also reports the affected sample, row, guide, or gene when applicable.

Message or condition	Meaning and safe action
`-c and -x are required`	Sample groups were omitted. Supply both lists from the header.
`unknown option`	The option is not supported. Use section 23 or `--help`. Do not substitute flags from another program.
`duplicate option`	An option was supplied twice. Keep one intended value.
`requires a value`	An option lacks its argument. Supply the correct value before another option.
`--threads must be >= 1`	Zero or a negative worker count was supplied. Use a positive integer.
`missing selected sample`	A selected name is not in the header. Check spelling, case, and spaces after commas.
`empty, duplicate or overlapping`	A group is empty or a sample is repeated. Correct the study's sample lists.
`paired arms must have equal nonzero sizes`	Paired lists have different sizes. Supply true pairs, or use unpaired mode only if the design calls for it.
`width ... expected ...`	A row has too many or too few tab fields. Repair the source export and check the row.
`invalid finite nonnegative count`	A selected value is negative, missing, infinite, undefined, or not numeric. Check the original measurement or count-generation step.
`total must be finite and positive`	A selected sample total is zero or invalid. Check counts and exclusions. Do not add arbitrary counts.
`local window` or `local SD`	The chosen window or local variation is undefined for this input. Check guide count, design, settings and data variation. A smaller window is not always valid.
`fewer than two retained genes`	Gene filtering leaves too few genes to standardize. Review input and `--minobs`.
`gene and guide destinations must differ`	Both results target the same file. Choose separate paths.
`I/O` or `output`	A path, permission, or storage operation failed. Check the parent folder, free space, and access. Keep old results and use a fresh folder.

Usage errors return exit code 2. Data, design, numerical, and file errors return 1. Success returns 0. An exit code is the program's completion status. `-q` does not hide errors. The options `-h` and `--help` show help successfully; `-V` and `--version` show the version successfully.

21. Reproducibility

Keep the original counts, exact sample lists, command, ZRust version, and source commit. Record the operating system, Rust version, analysis settings, and output names. A commit identifies one source snapshot. Do not put confidential data or credentials in a public issue.

Use this command to record the current source identity:

```
git rev-parse HEAD
```

The result is a hexadecimal identifier. It will depend on the source snapshot you obtained. Keep `Cargo.lock` unchanged when reproducing a result. The `--locked` build uses its exact dependency choices.

The source includes optional checks. Python 3 must be available for the fixture check and its integration test. A fixture is a small stored input or expected result used to check software behavior.

```
cargo fmt --check
cargo test --locked
python3 tools/release/verify_fixtures.py
```

Expected result: successful formatting and tests, then confirmation of 117 stored TSV fixture identities. These checks do not rerun published-screen benchmarks or establish support for every study.

22. Current limits

Source use is qualified only for macOS ARM64. Intel Mac, Linux and Windows are not qualified. A build on another platform does not establish scientific support. The largest tested input contained one million guides and 250,000 genes. Separate tests checked unequal-group unpaired behavior. Do not assume the largest matrix had unequal group sizes.

ZRust was 10.8 to 12.3 times faster than pinned Python DrugZ on three tested published olaparib screens. That comparison used serial output formatting on the tested macOS ARM64 workstation. It is not a universal speedup claim. Worker benefits depend on workload. Scientific calculations remain single-threaded.

This manual does not supply a binary, a registered package, a new biological finding, or a guarantee for arbitrary matrix sizes. It describes the tested source workflow. Read the manuscript for scientific methods and performance limits.

23. Complete command reference

The command name is `zrust`. The examples use `target/release/zrust` so the shell can find the local build. The minimum analysis form requires `-c LIST` and `-x LIST`. Supply `-i FILE` when you do not want standard input.

Option	Accepted value and behavior
`-i FILE`	UTF-8 TSV input. Omit or use `-` for standard input.
`-o FILE`	Gene output. Omit or use `-` for standard output.
`-f FILE`	Optional guide file. Here `-` is a literal filename, not standard output.
`-c LIST`	Required nonempty comma-separated control names. No whitespace trimming.
`-x LIST`	Required nonempty comma-separated treatment names. No whitespace trimming.
`-r LIST`	Exact gene names to exclude, separated by commas. Exclusion precedes normalization.
`-p INTEGER`	Normalized-count pseudocount. Default 5; minimum 0.
`-I INTEGER`	Guide index column. Only 0 is supported.
`--minobs INTEGER`	Minimum nonzero guide observations per gene. Default 1; minimum 1.
`--half_window_size INTEGER`	Local half-window. Default 500; minimum 1; input must also support the method's window rules.
`--threads INTEGER`	Output-format worker limit. Default 8; minimum 1; at most 8 are used.
`-q`	Suppress progress. Errors still print.
`-unpaired`	Compare means within the selected groups. Default mode is paired.
`-h`, `--help`	Print help and exit successfully.
`-V`, `--version`	Print version and exit successfully.

Do not repeat options. Do not add spaces around commas. Numeric options require integers within the parser's supported range. Oversized integers are errors. Paths with spaces must be quoted. Do not use `--threads=8`; provide the option and value as separate arguments.

No subcommand, configuration file, compressed-input option, or built-in threshold filter is implied. Choose the exact supported options above. Gene results contain both directional statistics regardless of which direction you plan to interpret.

24. Worked examples

These independent runs use fresh output folders. Run them from the repository root after building. If a folder already exists, choose a new name consistently in all commands. Do not delete an earlier result merely to reuse its name.

Paired: two controls and two treatments

```
mkdir results-worked-paired
target/release/zrust -i examples/paired/counts.tsv -c C1,C2 -x T1,T2 \
  --half_window_size 5 -q \
  -o results-worked-paired/genes.tsv -f results-worked-paired/guides.tsv
head -n 2 results-worked-paired/genes.tsv
head -n 2 results-worked-paired/guides.tsv
```

Expected first gene: `G18`. Its gene statistics start `sumZ=-11.03`, `numObs=8`, `normZ=-2.18`. The first guide is `g000`, gene `G00`. For the first pair, it has raw control count `1000`, `eb_std_0=0.193`, `zscore_fc_0=-1.381`, and `fc_0=-0.267`.

Unpaired: two controls and one treatment

```
mkdir results-worked-unpaired
target/release/zrust -i examples/unpaired/counts.tsv -c C1,C2 -x T1 \
  -unpaired --half_window_size 5 --minobs 1 -q \
  -o results-worked-unpaired/genes.tsv \
  -f results-worked-unpaired/guides.tsv
head -n 2 results-worked-unpaired/genes.tsv
head -n 2 results-worked-unpaired/guides.tsv
```

Expected first gene: `G18`. Its gene statistics start `sumZ=-12.87`, `numObs=4`, `normZ=-3.10`. The first guide is `g000`, gene `G00`. It has `raw_control_mean=1000.000`, `raw_treatment_mean=828.000`, `eb_std=0.191`, `zscore=-1.350`, and `mean_fc=-0.258`.

The input copies are intentionally identical. The selected groups and analysis mode differ. The unused `T2` column remains in the unpaired teaching file. Read DrugZ and the ZRust manuscript before interpreting biological data. ZRust-authored material uses the MIT license. Keep the source's third-party notices with it.